

Finding and Following Platelets – A guide to the software

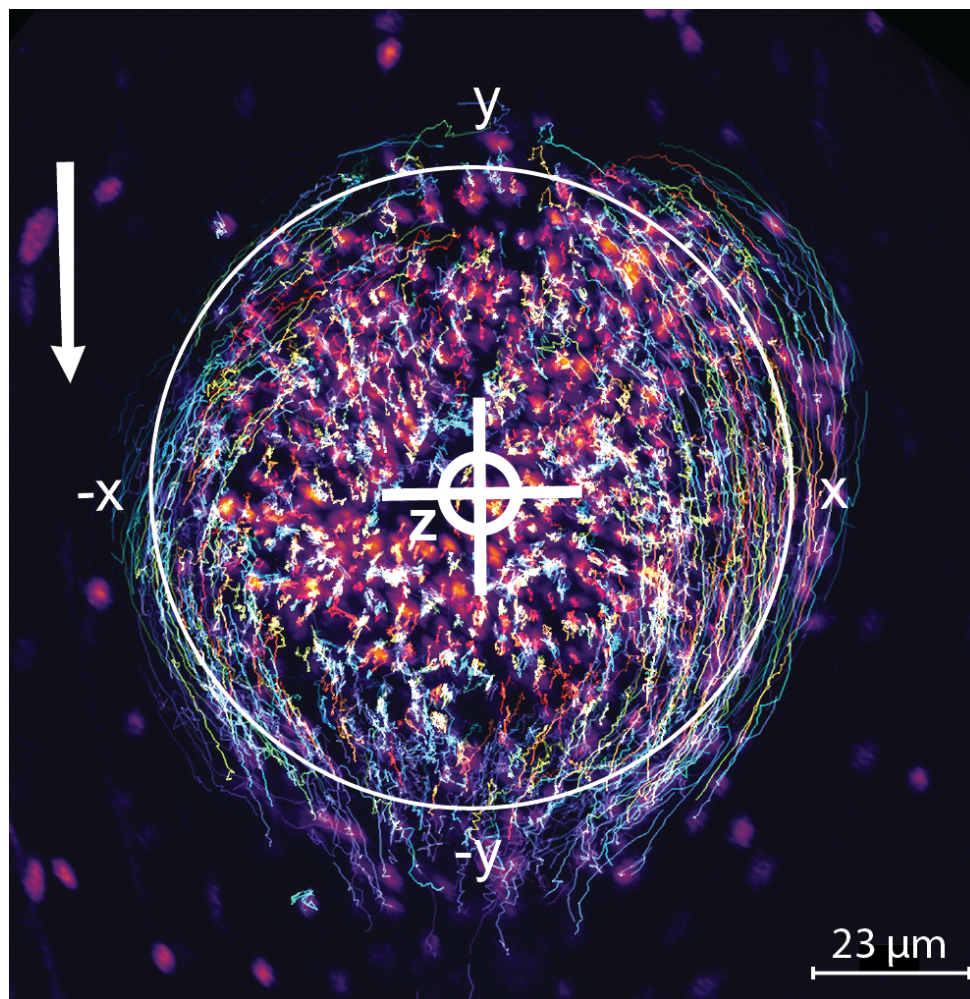


TABLE OF CONTENTS

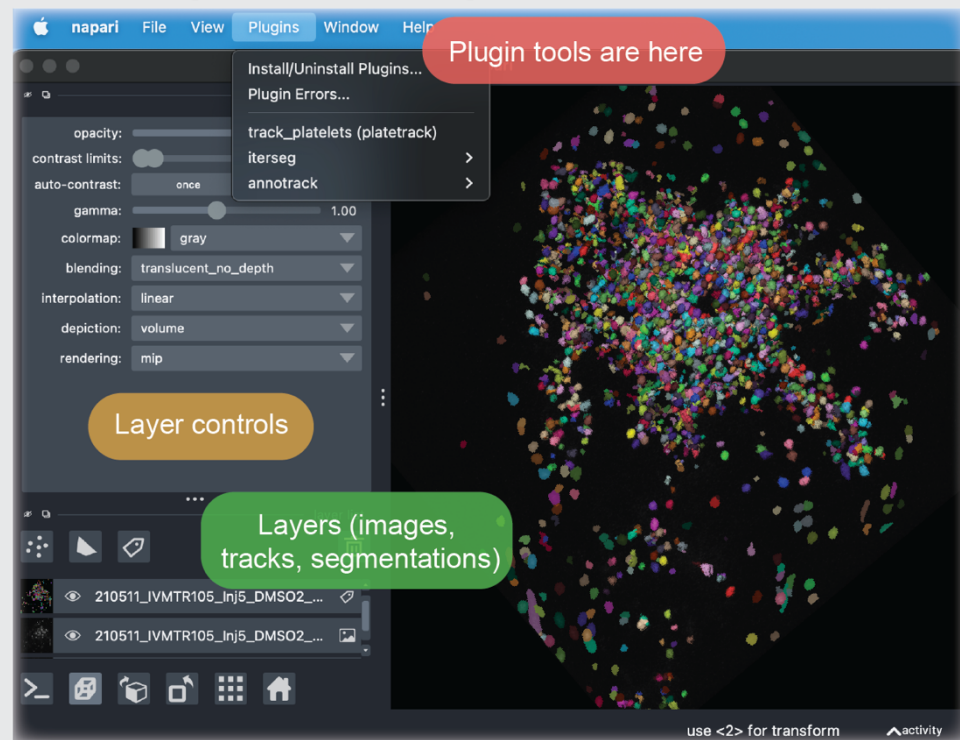
FINDING AND FOLLOWING PLATELETS – A GUIDE TO THE SOFTWARE	<u>1</u>
PACKAGE INSTALLATION	2
DOWNLOADING THE SAMPLE DATA	4
SEGMENTING SINGLE IMAGES USING ITERSEG	6
SEGMENTATION AND TRACKING WITH ITERSEG AND PLATETRACK	7
ANNOTATING TRACKS WITH ANNOTRACK	9
TRAINING A U-NET FOR AFFINITIES-BASED SEGMENTATION	12

Package installation

Our software depends on the **napari** viewer and the plugins **iterseg**, **platetrack**, and **annotrack**, all of which are available on PyPI, the Python Package Index. We describe below some basic commands for creating a new environment to run the software, but an extensive guide is out of the scope of this document. For in-depth introductions to Python virtual environments, see [here](#), [here](#), or [here](#). The packages can also be installed from the plugin installer in the still-experimental napari bundled app, which can be downloaded from the "Assets" drop down in the napari [releases page](#). For more on napari installation, see the [napari documentation](#).

1. Make sure your computer has an environment manager such as *anaconda* (<https://www.anaconda.com/download>) or (our recommendation) *miniforge* (<https://github.com/conda-forge/miniforge>).
2. Open terminal (if you are using MacOS or Ubuntu) or anaconda prompt or similar command line (if you are using Windows).
3. To create a new environment, type: `conda create -name p10 python=3.10` and press enter. We want to create a new environment because some of the packages we depend on (e.g., PyTorch) don't always play well in the sandpit with other packages that could be installed in an existing environment. This sort of interaction could cause errors when running the program. To activate the environment type: `conda activate p10` then press enter.
4. To install the plugins, type: `python -m pip install "napari[pyqt5]" iterseg platetrack annotrack` and press enter.
5. To open napari, type: `napari` and press enter. The graphical user interface will look as shown below. You can access the plugins through the plugins menu. When you hover over each plugin, you can see any functions it offers.

Napari-based Graphical User Interface



Downloading the sample data

Our sample data can be downloaded at the following link:

<https://dx.doi.org/10.26180/25137497>

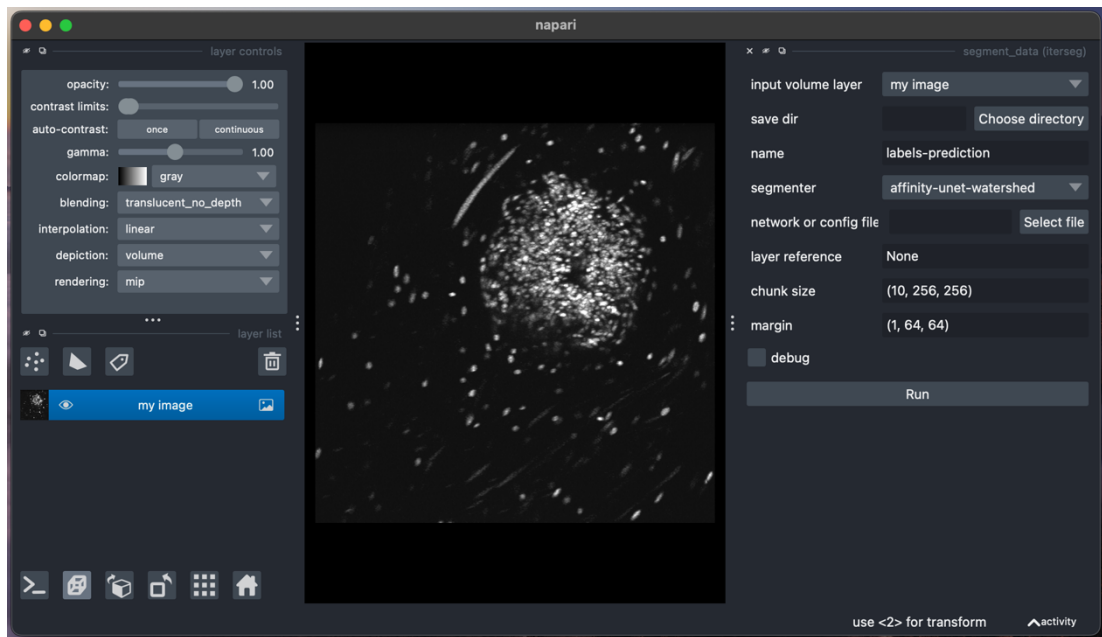
A description of the folder structure and contents is show below (folders are in **BOLD**; note that the description goes over the page):

- **Segmentation**
 - **exvivo_example**
 - **ground_truth**: 3D ground truth segmentations for individual frames of *ex vivo* data. Data are of dimensions (x, y, z = 512, 512, 33). The last two letters in the file name correspond to the initials of the proof-reader. The number in the file name corresponds to the number of the
 - **images**: 3D images corresponding to the *ex vivo* ground truth data for individual image frames. Images show the platelet surface marker. Data are of dimensions (x, y, z = 512, 512, 33).
 - **invivo_example**
 - **ground_truth**
 - **large – DMSO**: 3D ground truth segmentations for individual frames of data from a large DMSO treated thrombus *in vivo*. Data are of dimensions (x, y, z = 512, 512, 33).
 - **medium – control**: 3D ground truth segmentations for individual frames of data from a medium size control thrombus *in vivo*. Data are of dimensions (x, y, z = 512, 512, 33).
 - **small – cangrelor**: ground truth segmentations for individual frames of data from a small cangrelor treated thrombus *in vivo*. Data are of dimensions (x, y, z = 512, 512, 33).

- **images:** images for individual frames of data from the small, medium, and large thrombi described above. Images show the platelet surface marker. Data are of dimensions (x, y, z = 512, 512, 33).
- **training_data**
 - **training_gt:** A series of ground truth segmentations which can be used for training a network. In this folder, there are three labels from a early in an *ex vivo* flow experiment and seven images from *in vivo* experiments.
 - **training_images:** A series images which can be used, in conjunction with the ground truth segmentations above, for training a network. The images show platelet surface marker.
- **Tracking**
 - **human_exvivo:** contains two 10-minute time series featuring thrombi formed from human whole blood growing under different flow conditions (3000 s^{-1} and 600 s^{-1}). These images have four channels: (1) a calcium marker, (2) a fibrin marker, (3) a platelet surface marker, and (4) a TD image. The platelet surface marker channel is the one that should be used for segmentation.
 - **mouse_exvivo:** contains two 10-minute time series featuring thrombi formed from mouse whole blood growing under different flow conditions (1800 s^{-1} and 600 s^{-1}). These images have four channels: (1) a calcium marker, (2) a fibrin marker, (3) a platelet surface marker, and (4) a TD image. The platelet surface marker channel is the one that should be used for segmentation.
 - **mouse_invivo:** contains two 10-minute time series featuring thrombi formed from mouse whole blood growing under different flow conditions (1800 s^{-1} and 600 s^{-1}). These images have four channels: (1) a calcium marker, (2) a p-selectin marker, (3) a platelet surface marker, and (4) a TD image. The platelet surface marker channel is the one that should be used for segmentation.

Segmenting single images using Iterseg

1. The data are in the emerging standard [ome-ngff](#) format version 0.4. You can open each dataset by using the File > Open Folder menu selection in napari and selecting a folder with the extension `.ome.zarr`. Alternatively, you can drag and drop an `ome.zarr` folder onto napari. Be sure to select "iterseg", **not** napari builtins, as the reader plugin. Once the image is loaded you can adjust the contrast on each channel using the napari controls, and turn on 3D view by pressing the square/cube icon in the bottom left corner of the viewer.
2. To use iterseg's automatic segmentation, select Plugins > iterseg > ***segment_data*** from the top menu bar. If the loaded image was multi-channel, make sure the input volume layer is set to the channel containing the labelled platelets staining. (This is a subset of the platelets to make individual platelets easier to segment.) Choose a directory into which you want to save the segmentation by clicking ***choose directory*** and choose a name for the save file (you do not need to include an extension; `.ome.zarr` will be added automatically if it is missing). Make sure the segmenter is set to ***affinity-unet-watershed***. You do not need to choose a network or configuration file; for now we will use the default version. As we train the network on extra data, we will make other unets available. Layer reference should be None, chunk size should be (10, 256, 256) (this is specific to each particular unet) and margin should be (1, 64, 64) (this is the overlap between chunks processed by the unet; (0, 0, 0) will be faster but less accurate at the seams). Debug should be unticked. To segment the image, click ***Run***.

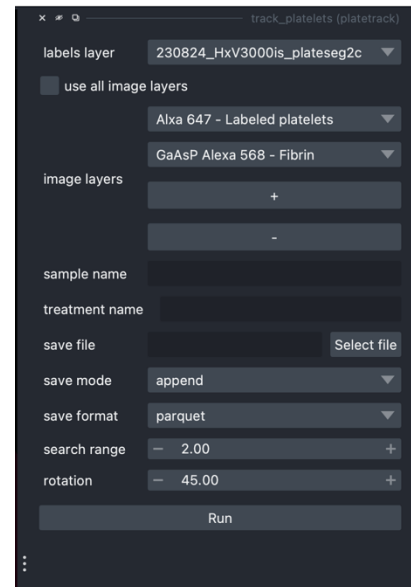


Segmentation and tracking with Iterseg and Platetrack

Note: pytorch on the CPU is quite slow. Segmenting a full time series on an Apple MacBook Pro M1 Max takes around 50h. If you have a computer with an NVIDIA or AMD GPU, you can install the corresponding version of PyTorch by following the CUDA or ROCm (respectively) installation instructions for your operating system on pytorch.org. On a Linux computer, the default installation from PyPI should Just Work™, but one never knows with PyTorch. 🙌

1. To segment a full timeseries (video of a thrombus) with Iterseg, follow the same process as above but choose one of the full timeseries from one of the tracking folders. Repeat step 2 from above ensuring to choose the channel with the marker for all the platelets. This process may take several minutes to several hours depending on your computer.

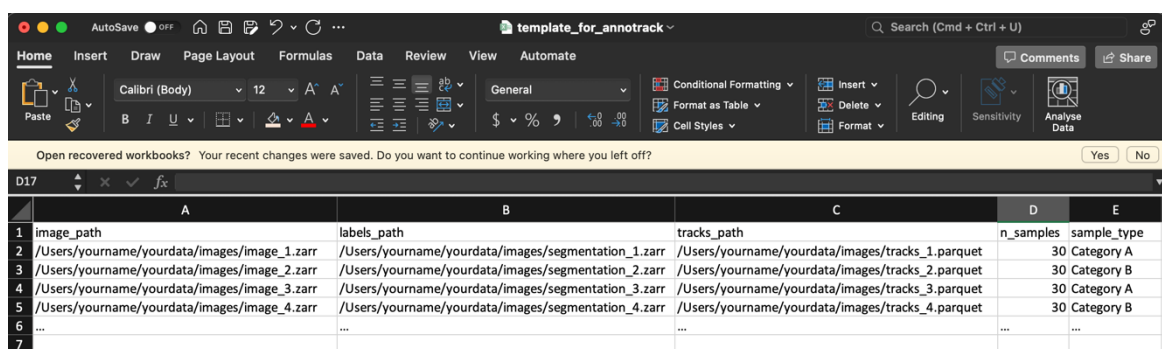
2. Once the segmentation is complete, open the ***track_platelets (platetrack)*** widget. Choose the segmentation layer as the labels layer. You can measure fluorescence across all image layers by ticking the "use all image layers" box, or you can untick it and select individual image layers (add more layers with the + button, remove them with the - button). Type a sample name and a treatment name. Select a file to save the tracks to. We recommend the [parquet format](#), which preserves more information and uses less storage space than csv. However, if you want to open your tracks data in a spreadsheet program, use csv. The search range is the radius in microns in which to look for a platelet in the next frame. Rotation applies a rotation to the image to align the y axis with the blood flow. 45° is the correct setting for our data. Click ***Run***.



Annotating tracks with Annotrack

Annotrack helps you quantify how well your automatic tracking worked *without* requiring dense ground truth. It does this by sampling from individual tracks and letting you inspect them individually, keeping track of any errors and error types.

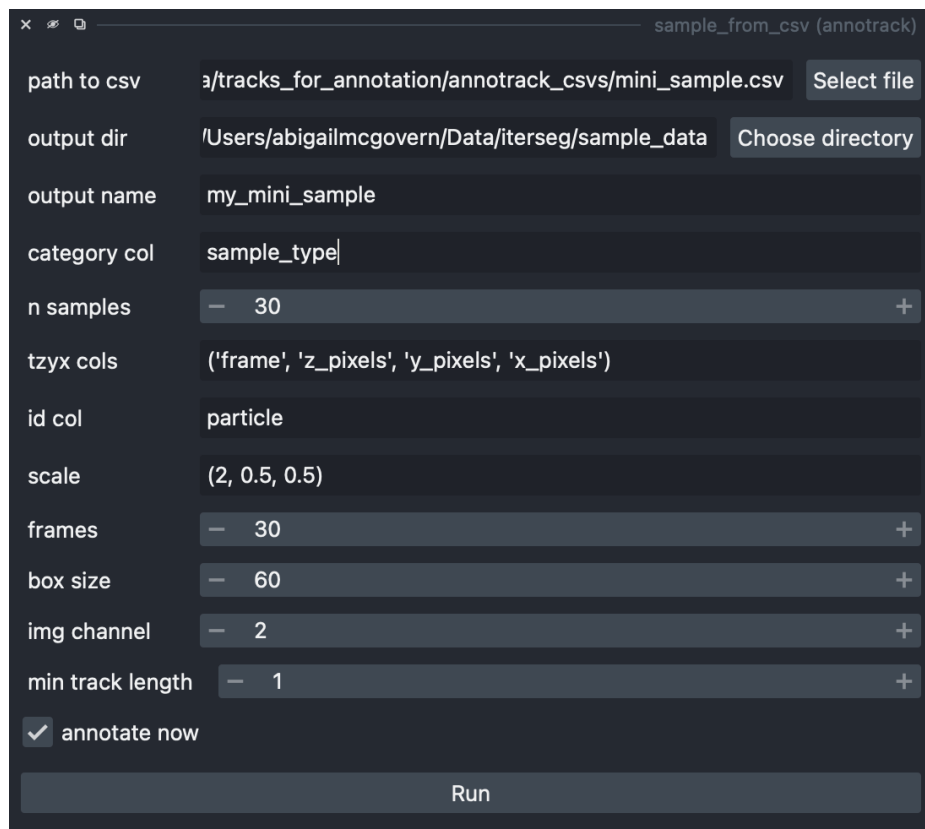
1. To generate track samples and annotate them, you will need to first generate a csv file (comma separated variable file) with the paths to the images, tracks, and labels in the columns. A good way to make this csv is by using excel to make a spreadsheet (example below) and save as a csv. The example below shows the required columns for the csv. **We you can find a template csv at the link provided** (<https://gist.github.com/AbigailMcGovern/975ed4dd686971c99ca6970f12e6f0bb>), **which you can download and fill in any spreadsheet program**, such as Microsoft Excel, Apple Numbers, or LibreOffice Calc. The first three columns are called *image_path*, *labels_path*, *tracks_path*, and these must contain the full file path to the corresponding data. (Try to avoid any special characters when naming the folders and files – this will help avoid errors.) The next columns *n_samples* and *sample_type* allow you to specify the number of track samples to take from each file and the category (e.g., treatment, specific antibody, shear rate, laser power, you name it), respectively. Including data from each experimental condition can help you understand whether different experimental conditions systematically affect tracking accuracy. Note: The *n_samples* column is optional (see step 2).



	A	B	C	D	E
1	image_path	labels_path	tracks_path	n_samples	sample_type
2	/Users/yourname/yourdata/images/image_1.zarr	/Users/yourname/yourdata/images/segmentation_1.zarr	/Users/yourname/yourdata/images/tracks_1.parquet	30	Category A
3	/Users/yourname/yourdata/images/image_2.zarr	/Users/yourname/yourdata/images/segmentation_2.zarr	/Users/yourname/yourdata/images/tracks_2.parquet	30	Category B
4	/Users/yourname/yourdata/images/image_3.zarr	/Users/yourname/yourdata/images/segmentation_3.zarr	/Users/yourname/yourdata/images/tracks_3.parquet	30	Category A
5	/Users/yourname/yourdata/images/image_4.zarr	/Users/yourname/yourdata/images/segmentation_4.zarr	/Users/yourname/yourdata/images/tracks_4.parquet	30	Category B
6	...	...	...	...	...
7					

2. Once you have created the csv, you can load it using the annotrack widget ***sample_from_csv*** and fill in the fields (as shown below). In the first field, click **Select file** to choose the csv file you just created. In the next click **Choose directory** to choose the folder into which to save both the sampled tracks and the track

annotations. Now you can give the output file a name (this is the file into which both your annotations will be saved). Leave the



sample_from_csv (annotrack)

path to csv: a/tracks_for_annotation/annotrack_csvs/mini_sample.csv [Select file]

output dir: /Users/abigailmcgovern/Data/iterseg/sample_data [Choose directory]

output name: my_mini_sample

category col: sample_type

n samples: 30

tzyx cols: ('frame', 'z_pixels', 'y_pixels', 'x_pixels')

id col: particle

scale: (2, 0.5, 0.5)

frames: 30

box size: 60

img channel: 2

min track length: 1

☒ annotate now

Run

category col as *sample_type* (unless you changed it in the csv: they need to match). Leave the *n samples* box as 30 unless you have (1) not included the column in your csv and (2) want to take a different number of samples from each experiment. If you have included the *n_samples* column in your csv this will override the *n samples* box here. Leave *tzyx cols* as *("frame", "z_pixels", "y_pixels", "x_pixels")* and leave *id col* as *particle* unless you have changed column names in your tracks files. These values correspond to the names of the columns containing information about frame, coordinates in pixels, and platelet ID (and the order is important). Annotrack does not yet read pixel scale metadata so it must be set manually in the *scale* box as a tuple of numbers. This is for visualisation only, so approximate values are fine — (2, 0.5, 0.5) is a suitable value for our sample data (in order "zyx"). *frames* determines the number of frames to display for each sampled track; *box size*, the size (in pixels along the highest-resolution axes) of the crop around the track data; and *img channel* the channel number containing the platelet surface marker for the image data we

want to view. The default values are good for our data. *Min track length* is the minimum track length of platelets you will be shown. *Min track length* of 1 indicates you will look at all tracked platelets. However, if you are interested in only looking at longer tracks (i.e., stably incorporated platelets), select a higher number.

3. Click Run to annotate the tracks. To annotate the tracks use the following key strokes:

- '2' - move to next sample
- '1' - move to previous sample
- 'y' - annotate as correct (will move to the next sample automatically)
- 'n' - annotate as containing an error (will move to the next sample automatically)
- 'i' - annotate the frame following a ID swap error
- 't' - annotate the fame following an incorrect termination
- 'Shift-t' - annotate the frame containing a false start error
- 's' - annotate an error ('i', 't', or 'Shift-t') as being associated with a segmentation error (merge or split of objects)

Note: due to limitations in how napari handles keystrokes, the "1" and "2" shortcuts won't work if a layer is selected. Click on an empty area of the layer list to deselect all layers before using these shortcuts.

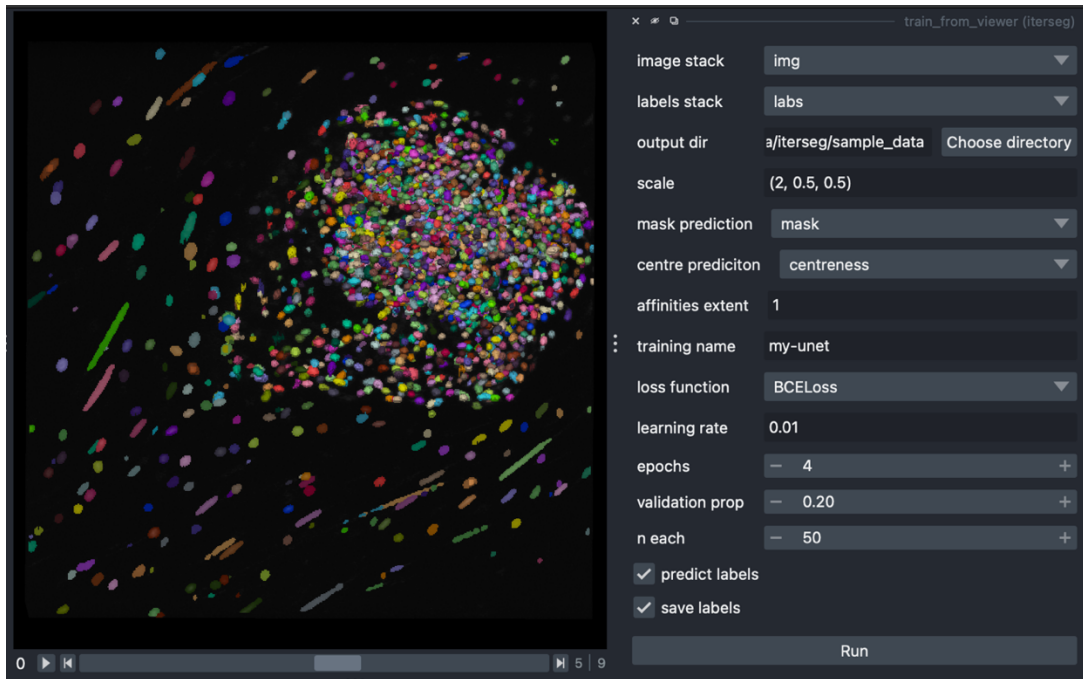
When an error is associated the specific frame ('i', 't', 'Shift-t', or 's'), the frame number (within the original image) will be added to a list of errors for the sample within the sample's (.smpl) info data frame. E.g., you may have a list of ID swaps for your sampled track segment ([108, 111, 112]) and a corresponding list of segmentation error associations ([108, 112]).

Training a U-net for affinities-based segmentation

1. Before trying to train a new neural network on your own data, we recommend you run through training one on our sample data. The data with which to train the network can be found in the **segmentation > training_data** folder. Use the **load_data** widget (as you would when segmenting data) to load in the stack of training images and ground truth segmentations. Load images from the **segmentation > training_data > training_images** folder, then load the corresponding ground truth from the **segmentation > training_data > training_gt** folder.
2. Once you have loaded in the data you will be able to train the network using the **train_from_viewer** widget (shown below). Once you have loaded the widget first make sure the correct image and labels layers are selected. Then choose a directory into which to save output using **Choose directory** (includes: checkpoints, the unet, training chunks, computed training labels, etc). Set the **scale** to (2, 0.5, 0.5). The **mask prediction** allows you to choose what type of feature channel is fed to the segmentation algorithm to find platelet foreground pixels – **mask** is the best option. The **centre prediction** function allows you to choose what type of feature channel is fed to the segmentation algorithm to find the centre of platelets. For this I would suggest **centreness**, although **centreness-log** produces similar results. Set **affinities extent** to 1. Give your output U-net a name in **training name**. The remaining parameters determine the training program used. Use **BCELoss** as the loss function for this data. Default parameters are **learning rate** = 0.01, **epochs** = 4, **validation split** = 0.2 (20%), and **n each** = 50. Learning rate determines how much the values in the network (which determine its output) are updated (corrected with respect to the ground truth) with every step of training. Epochs refers to the number of times you cycle through the data. Validation split determines how much of the data is used for validation versus learning (usually 20%). N each is how many subsamples are taken and augmented from the training data (n each x n samples = mini batch size – in this case 450). If you have **predict labels** ticked, then the trained network will be used to produce a segmentation of the training data once training is done (this will be displayed as a new layer). If you have **save labels** ticked the newly predicted labels

will be saved. Whilst predicting on training data is not a particularly valid way to assess segmentation quality, it will give you a good indication of how the network is performing.

3. Once you are happy with the parameters, click **Run**. Once running, the results of training will be printed in the command line or terminal you used to launch napari.



The command line will show you (1) the progress through training (might take several hours depending on your computer), and (2) the training and validation loss throughout training.

4. In the output in the directory you choose, you will be able to open a text file called unet_paths.txt. This is a plain text file that will record the file paths to the u-net from any training session for which that output directory was chosen. This will tell you where the unet is. The file path for the unet can be pasted into (or found using the **Choose file** button) the **unet or config file** field of the **segment data** widget to use the unet to segment with.